



Tutorial 2: Training and Serving ML Model in Docker (with Flask)

Learning Goals

By the end of this class we will be able to:

- Package an ML training script inside a container
 - Save a trained model to disk from inside Docker
 - Serve predictions through a simple Flask API
 - Test our containerised API from the host machine
-

0. Recap

In the previous lesson, we learned:

- What **containers** are and how they differ from VMs.
- How to install and run Docker.
- How to manage containers (`docker ps` , `docker stop` , `docker rm` , `docker logs`).
- How to use **volumes** to persist data.
- How to build our own **image** using a **Dockerfile**.

👉 Today, we go one step further: we will train a **Machine Learning model inside a Docker container** and then serve it with a **Flask API**.

1. Project Structure

We will use the following files:

```
.
├── app.py          # Flask API to serve predictions
├── train.py        # Script to train the model
├── requirements.txt # Python dependencies
├── Dockerfile      # Defines container image
├── run.sh          # Script to train model + run API
└── test_client.py  # Exercise: Script to test API requests
```

2. Training Script

train.py

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
import joblib

# Load dataset
X, y = load_iris(return_X_y=True)

# Train simple model
model = LogisticRegression(max_iter=200)
model.fit(X, y)

# Save model
joblib.dump(model, "model.pkl")
print("✅ Model trained and saved as model.pkl")
```

This script trains a simple classifier on the Iris dataset and saves it as `model.pkl`.

3. Serving Script (Flask API)

app.py

```
from flask import Flask, request, jsonify
import joblib
import numpy as np

# Load trained model
model = joblib.load("model.pkl")

app = Flask(__name__)

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["features"]
    prediction = model.predict([np.array(data)])
    return jsonify({"prediction": int(prediction[0])})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

This API exposes one endpoint `/predict` where you send a JSON with `"features"`.

4. Dependencies

requirements.txt

```
flask
scikit-learn
joblib
numpy
```

5. Run Script

run.sh

```
#!/bin/bash
# First, train the model
python train.py
# Then, start Flask API
python app.py
```

Make it executable:

```
chmod +x run.sh
```

6. Dockerfile

Dockerfile

```
FROM python:3.10

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["bash", "run.sh"]
```

7. Build the Image

```
docker build -t ml-flask-app .
```

8. Run the Container

```
docker run -p 8000:8000 ml-flask-app
```

What happens?

- The container executes `run.sh` :
 1. Trains the model (`train.py`)
 2. Starts the Flask API (`app.py`)

✅ The API is available on <http://localhost:8000>

9. Test the API

Use `curl` :

```
curl -X POST -H "Content-Type: application/json" -d '{"features": [5.1, 3.5, 1.4, 0.2]}' http://localhost:8000/predict
```

✅ Expected output:

```
{
  "prediction": 0
}
```

10. Implement a Test Client

So far, we've tested the API using curl. Now let's **build a Python script** that does the same thing.

👉 Create a file called `test_client.py` .

Our task:

- Use the **requests** library in Python.
- Send a POST request to `http://localhost:8000/predict` .

- Include a JSON body with `"features": [5.1, 3.5, 1.4, 0.2]` .
- Print the response from the API.



Hints:

- The `requests` library has a function `requests.post(url, json=...)` .
- We can then call `.json()` on the response to see the server reply.

✓ **Expected behaviour:** When we run our script, we should get the same kind of result as with `curl` , e.g. `{"prediction": 0}` .

11. Reflection

- What happens when the container starts?
- Why do we need to install dependencies inside the image?
- Why do we expose port `8000:8000` ?
- Where is `model.pkl` stored? What happens if we delete the container?
- How could we persist the model using a **volume**?

12. Exercises

Now it's time to extend our system.

A. Persist the Model with a Volume

Right now, the trained `model.pkl` exists **only inside the container**. If the container is removed, the model file disappears.

Our task:

- Run the container with a **volume mount** so that `model.pkl` is stored on your host machine.
- Check that the file still exists even after you stop and remove the container.



Hint:

- Remember the `-v $(pwd)/data:/app/data` option from Lesson 1.
- Where would you need to save `model.pkl` so that it ends up in `/app/data` inside the container?

B. Add a Batch Prediction Endpoint

Our API currently makes predictions for **one input at a time**. In real systems, we often want to send **multiple samples in one request** (batch prediction).

Our tasks:

- Add a new endpoint `/prediction-batch` to `app.py`
- It should:
 - Accept a JSON body with `"features"` as a **list of lists** (multiple samples).
 - Return predictions for all of them.

Example request:

```
{
  "features": [
    [5.1, 3.5, 1.4, 0.2],
    [6.7, 3.0, 5.2, 2.3]
  ]
}
```

Expected response:

```
{
  "predictions": [0, 2]
}
```



Hint:

- `model.predict()` can take a list/array of multiple samples at once.
- How is this different from the single-sample case?



Wrap-up

- We trained an ML model **inside Docker**.
- We served it with a **Flask API**.
- We tested the API from our host machine.
- We extended it with:
 - **Volumes** to persist the trained model.
 - **Batch predictions** to support real-world usage.